LABORATORIO 10

Configuración de ambientes en Spring Boot

Desarrollo de Aplicaciones Web

Sección C24 - D
Estudiante: Jesús Castillo Sumire

Objetivo

Configurar una aplicación para trabajar con ambientes separados mediante Spring Profiles y variables de entorno, definiendo para cada uno su perfil, puerto y base de datos. También se implementa el CRUD REST del caso de empleados de TechSolutions SAC.

Cómo se trabajó

Preparé dos proyectos Maven independientes dentro de Semana 10: una copia de la API de productos de Semana 5, adaptada a perfiles, y una API de empleados desarrollada para el caso empresarial de la guía. Los valores de conexión se pueden cambiar sin editar el código fuente.

Herramientas y procedimiento

Visual Studio Code, Java 17, Spring Boot, Maven, Spring Profiles, Spring Data JPA/Hibernate, MySQL de XAMPP y Postman. El procedimiento consiste en crear las bases del ambiente, seleccionar un perfil en Run and Debug, iniciar la aplicación, consultar su endpoint de configuración y probar las operaciones CRUD.

Flujo de selección de ambiente

Estado de validación: ambos proyectos compilaron con Maven. La validación en ejecución contra MySQL y Postman no se pudo completar porque MySQL de XAMPP estaba detenido; el informe no presenta capturas ficticias.

1. Ambientes y perfiles

Se configuraron cuatro ambientes. Los perfiles no productivos usan `ddl-auto=update` para mantener el esquema durante el desarrollo; producción usa `validate` y requiere que el esquema exista previamente.

Perfil	Puerto	Base productos	Base empleados	Esquema
dev	8080	productos_dev	empleados_dev	update
qa	8082	productos_qa	empleados_qa	update
test	8084	productos_test	empleados_test	update
prod	8086	productos_prod	empleados_prod	validate

Configuración común de empleados

Este bloque define el perfil inicial predeterminado y permite reemplazar el puerto y la conexión desde variables del sistema. El valor de contraseña queda vacío para la configuración local de XAMPP; en otro entorno debe proporcionarse como variable y no guardarse en el repositorio.

```
spring.application.name=gestion-empleados-api
spring.profiles.active=${PROFILE_ACTIVE:dev}
server.port=${SERVER_PORT:8080}
spring.datasource.url=${DB_URL:jdbc:mysql://localhost:3306/empleados_dev?createDatabaseIfNotExist=
    true&useSSL=false&serverTimezone=UTC}
spring.datasource.username=${DB_USER:root}
spring.datasource.password=${DB_PASSWORD:}
spring.jpa.hibernate.ddl-auto=update
```

Archivos application-.properties

Los archivos application-dev.properties, application-qa.properties, application-test.properties y application-prod.properties contienen los valores específicos de cada ambiente. La configuración de Run and Debug en .vscode/launch.json establece PROFILE_ACTIVE, SERVER_PORT, DB_URL, DB_USER y DB_PASSWORD al iniciar cada perfil.

```
"name": "Empleados - QA",
"mainClass": "com.tecsup.empleados.EmpleadoApplication",
"env": {
  "PROFILE_ACTIVE": "qa",
  "SERVER_PORT": "8082",
  "DB_URL": "jdbc:mysql://localhost:3306/empleados_qa?createDatabaseIfNotExist=true&useSSL=false&s
    erverTimezone=UTC",
  "DB_USER": "root",
  "DB_PASSWORD": ""
}
```

2. API de productos copiada de Semana 5

El proyecto conserva las rutas CRUD de productos y añade un endpoint que permite comprobar qué ambiente está activo. El mismo patrón de perfiles está configurado con las bases productos_dev, productos_qa, productos_test y productos_prod.

Endpoint de diagnóstico de configuración

```
@GetMapping
public Map<String, String> obtenerConfiguracion() {
    String[] perfiles = environment.getActiveProfiles();
    String perfil = perfiles.length == 0 ? "default" : perfiles[0];
    String baseDatos = URI.create(urlBaseDatos.substring("jdbc:".length())).getPath();
    baseDatos = baseDatos == null ? "" : baseDatos.replaceFirst("^/", "");
    return Map.of("ambiente", ambiente, "perfil", perfil, "puerto", puerto,
            "baseDatos", baseDatos);
}
```

Este endpoint lee el perfil de Spring y propiedades ya resueltas, y devuelve un resumen sin exponer la contraseña de conexión. Se consulta en `GET /config`.

3. CRUD REST de empleados

La entidad Empleado se persiste en la base del perfil seleccionado. El DTO aplica validación de nombre, correo y sueldo; el repositorio extiende JpaRepository y el servicio concentra las operaciones y el caso de empleado inexistente.

Rutas principales

Método	Ruta	Función	Respuesta esperada
GET	/empleados	Listar empleados	200
GET	/empleados/{id}	Consultar uno	200 / 404
POST	/empleados	Crear	201 / 400
PUT	/empleados/{id}	Actualizar	200 / 400 / 404
DELETE	/empleados/{id}	Eliminar	204 / 404
GET	/configuracion	Ver ambiente activo	200

Controlador REST

El controlador traduce solicitudes HTTP a llamadas del servicio; @Valid activa las restricciones del DTO antes de guardar. La lógica no se mezcla con el acceso a base de datos.

```
@PostMapping
@ResponseStatus(HttpStatus.CREATED)
public Empleado crear(@Valid @RequestBody EmpleadoRequest request) {
    return service.crear(request);
}

@PutMapping("/{id}")
public Empleado actualizar(@PathVariable Long id,
        @Valid @RequestBody EmpleadoRequest request) {
    return service.actualizar(id, request);
}
```

4. Persistencia y validaciones

La entidad está mapeada a la tabla empleado. Las columnas obligatorias y la unicidad del correo se definen a nivel de persistencia; el DTO valida los datos entrantes antes de invocar el servicio. El script SQL crea las cuatro bases y prepara la tabla de producción, donde Hibernate valida el modelo al arrancar.

```
@Entity
@Table(name = "empleado")
public class Empleado {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    @Column(length = 100, nullable = false) private String nombres;
    @Column(length = 100, nullable = false) private String apellidos;
    @Column(length = 100, nullable = false, unique = true) private String email;
    @Column(precision = 10, scale = 2, nullable = false) private BigDecimal sueldo;
}
```

Las colecciones Lab10-Productos.postman_collection.json y Lab10-Empleados.postman_collection.json incluyen consultas de configuración y operaciones CRUD, además de escenarios de validación. La importación de las colecciones está preparada; no se afirma que hayan sido ejecutadas.

Pruebas recomendadas al iniciar MySQL

Caso	Solicitud	Resultado a comprobar
Perfil DEV	GET http://localhost:8080/configuracion	JSON muestra dev, 8080 y empleados_dev
Perfil QA	GET http://localhost:8082/configuracion	JSON muestra qa, 8082 y empleados_qa
Crear empleado	POST /empleados con JSON válido	201 y registro persistido
Correo inválido	POST /empleados con email incorrecto	400 de validación
Listar / consultar	GET /empleados y /empleados/{id}	200 y datos del empleado
Actualizar / eliminar	PUT y DELETE /empleados/{id}	200 al editar y 204 al eliminar

5. Inicio local en Visual Studio Code

1. Iniciar MySQL en XAMPP y confirmar el puerto 3306.
2. Abrir en VS Code la carpeta Semana10/api-productos-perfiles o Semana10/gestion-empleados-api.
3. Seleccionar Run and Debug y elegir DEV, QA, TEST o PROD.
4. Iniciar la opción elegida y consultar GET /config o GET /configuracion en el puerto asociado.
5. Importar la colección Postman del proyecto y ejecutar primero el endpoint de configuración; después probar el CRUD.
6. En producción, preparar el esquema antes de ejecutar porque el perfil usa validate.

6. Evidencias y estado de validación

La evidencia disponible en esta entrega es el código fuente del proyecto, sus configuraciones de perfil, las opciones de VS Code y las colecciones Postman incluidas. Los registros locales muestran compilación correcta con Maven en ambos proyectos.

Queda pendiente incorporar capturas reales de VS Code con cada perfil iniciado, XAMPP/phpMyAdmin mostrando las bases activas y Postman con las respuestas. Durante la revisión MySQL de XAMPP estaba detenido; por ello no se generaron esas capturas ni se presentan respuestas simuladas como si fueran pruebas reales.

Archivos principales

Proyecto	Ubicación dentro del repositorio
Productos por perfil	Semana10/api-productos-perfiles/
CRUD de empleados	Semana10/gestion-empleados-api/
SQL de bases de productos	api-productos-perfiles/database/productos_ambientes.sql
SQL de empleados	gestion-empleados-api/database/empleados_ambientes.sql
Colecciones Postman	postman/Lab10-*.postman_collection.json
Configuraciones VS Code	Cada proyecto/.vscode/launch.json

7. Observaciones

1. Al separar los perfiles pude definir distintos puertos y bases de datos sin mantener una sola configuración fija.

2. La variable PROFILE_ACTIVE me permite escoger el ambiente desde VS Code al iniciar la aplicación.

3. El endpoint de configuración ayuda a confirmar qué perfil, puerto y base se cargaron sin mostrar la contraseña.

4. En los ambientes de desarrollo, calidad y pruebas se usa update; en producción se usa validate para exigir que el esquema exista.

5. La compilación terminó correctamente, pero no pude comprobar las conexiones ni las respuestas de Postman porque MySQL de XAMPP estaba detenido.

8. Conclusiones

1. Con este laboratorio comprendí que Spring Profiles permite preparar una misma aplicación para varios ambientes y cargar valores distintos según el perfil activo.

2. Al usar variables de entorno puedo modificar la conexión, el usuario, la contraseña y el puerto sin cambiar las clases Java.

3. La configuración de Run and Debug en VS Code facilita iniciar cada perfil y reduce errores al cambiar de ambiente.

4. El endpoint de configuración es útil para revisar rápidamente si la aplicación tomó el perfil y la base de datos que esperaba.

5. La implementación y la compilación están listas; para dar por terminada la validación debo iniciar MySQL, ejecutar las colecciones Postman y guardar capturas reales de las respuestas.